Lab 6: Worst-fit Best-fit First-fit

code:

```
#include <stdio.h>

void firstFit(int blocks[], int m, int processes[], int n) {
    int allocation[n];

    for (int i = 0; i < n; i++)
        allocation[i] = -1;

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (blocks[j] >= processes[i]) {
                allocation[i] = j;
                blocks[j] -= processes[i];
                break;
            }
        }
    }

    printf("\nFirst Fit Allocation:\n");
    for (int i = 0; i < n; i++) {
        if (allocation[i] != -1)
            printf("Process %d -> Block %d\n", i + 1, allocation[i] + 1);
        else
            printf("Process %d -> Not Allocated\n", i + 1);
    }
}

void bestFit(int blocks[], int m, int processes[], int n) {
    int allocation[n];

    for (int i = 0; i < n; i++)
        allocation[i] = -1;

    for (int i = 0; i < n; i++) {
        int bestIndex = -1;

        for (int j = 0; j < m; j++) {
```

```
            if (blocks[j] >= processes[i]) {
                if (bestIndex == -1 || blocks[j] < blocks[bestIndex])
                    bestIndex = j;
            }
        }

        if (bestIndex != -1) {
            allocation[i] = bestIndex;
            blocks[bestIndex] -= processes[i];
        }
    }

    printf("\nBest Fit Allocation:\n");
    for (int i = 0; i < n; i++) {
        if (allocation[i] != -1)
            printf("Process %d -> Block %d\n", i + 1, allocation[i] + 1);
        else
            printf("Process %d -> Not Allocated\n", i + 1);
    }
}

void worstFit(int blocks[], int m, int processes[], int n) {
    int allocation[n];

    for (int i = 0; i < n; i++)
        allocation[i] = -1;

    for (int i = 0; i < n; i++) {
        int worstIndex = -1;

        for (int j = 0; j < m; j++) {
            if (blocks[j] >= processes[i]) {
                if (worstIndex == -1 || blocks[j] > blocks[worstIndex])
                    worstIndex = j;
            }
        }

        if (worstIndex != -1) {
            allocation[i] = worstIndex;
            blocks[worstIndex] -= processes[i];
        }
    }

    printf("\nWorst Fit Allocation:\n");
```

```
    for (int i = 0; i < n; i++) {
        if (allocation[i] != -1)
            printf("Process %d -> Block %d\n", i + 1, allocation[i] + 1);
        else
            printf("Process %d -> Not Allocated\n", i + 1);
    }
}

int main() {
    printf("usn:1WA24CS235\n");
printf("name:Riya gupta\n");
    int m, n;

    printf("Enter number of memory blocks: ");
    scanf("%d", &m);

    int blocks[m], blocks_copy1[m], blocks_copy2[m];

    printf("Enter sizes of memory blocks:\n");
    for (int i = 0; i < m; i++) {
        scanf("%d", &blocks[i]);
        blocks_copy1[i] = blocks[i];
        blocks_copy2[i] = blocks[i];
    }

    printf("Enter number of processes: ");
    scanf("%d", &n);

    int processes[n];

    printf("Enter sizes of processes:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &processes[i]);
    }

    firstFit(blocks, m, processes, n);
    bestFit(blocks_copy1, m, processes, n);
    worstFit(blocks_copy2, m, processes, n);

    return 0;
}
```

output:

Output

```
usn:1WA24CS235
name:Riya gupta
Enter number of memory blocks: 5
Enter sizes of memory blocks:
400
700
200
300
600
Enter number of processes: 4
Enter sizes of processes:
212
512
312
526

First Fit Allocation:
Process 1 -> Block 1
Process 2 -> Block 2
Process 3 -> Block 5
Process 4 -> Not Allocated

Best Fit Allocation:
Process 1 -> Block 4
Process 2 -> Block 5
Process 3 -> Block 1
Process 4 -> Block 2
```

```
Worst Fit Allocation:
Process 1 -> Block 2
Process 2 -> Block 5
Process 3 -> Block 2
Process 4 -> Not Allocated


=== Code Execution Successful ===
```